DeHacker

Code Security Assessment

Mapping Token V2 (MAPO ERC20/BEP20)

May 25th, 2026

Contents

Summary

DeHacker's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best prac tices. Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow/underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service/logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Issue Categories

Every issue in this report was assigned a severity level from the following:

Critical severity issues

A vulnerability that can disrupt the contract functioning in a number of scenarios or creates a risk that the contract may be broken.

Major severity issues

A vulnerability that affects the desired outcome when using a contract or provides the opportunity to use a contract in an unintended way.

Medium severity issues

A vulnerability that could affect the desired outcome of executing the contract in a specific scenario.

Minor severity issues

A vulnerability that does not have a significant impact on possible scenarios for the use of the contract and is probably subjective.

Informational

A vulnerability that has informational character but is not affecting any of the code.

Overview

Project Summary

Project Name	Mapping Token V2 (MAPO ERC20/BEP20)
Platform	EVM (BSC & Ethereum)
Website	https://www.mapprotocol.io/
Type	Token
Language	Solidity
Codebase	https://github.com/mapprotocol/mapping-token-contracts/blob/main/contracts/MappingTokenV2.sol
Token Address	0x7046933234A82AF77F14625e8d0fA9Bcc5044a7E (ERC20) 0x7046933234A82AF77F14625e8d0fA9Bcc5044a7E (BEP20)

Vulnerability Summary

Vulnerability Level	Total	Mitigated	Declined	Acknowledged	Partially Resolved	Resolved
Critical	0	0	0	0	0	0
Major	2	0	0	0	0	2
Medium	1	0	0	0	0	1
Minor	1	0	0	0	0	1
Informational	1	0	0	0	0	1
Discussion	0	0	0	0	0	0

Audit scope

ID	File	SHA256 Checksum
GLOBAL	https://github.com/mapprotocol/mapping-token-contracts/blob/main/contracts/MappingTokenV2.sol	808fa7da8f2ebd5b3e6027ad1dd6fec26ad6e61be92d9fbb3c7d6b4d5a1c3124

Findings

ID	Title	Severity	Status
GLOBAL-01	Centralization Related Risks	Major	Resolved
GLOBAL-02	Uncleared Post-Rotation Minter Allowances	Major	Resolved
GLOBAL-03	Missing Context in Mint Cap Events	Medium	Resolved
GLOBAL-04	Zero-Amount Mint and Burn Permitted	Minor	Resolved
GLOBAL-05	Deprecated ERC20Permit Import	Informational	Resolved

MAJOR

GLOBAL-01 | Centralization Related Risks

Issue	Severity	Status
Centralization Related Risks	Major	Resolved

Description

Some roles in the MappingTokenV2 contract have the following permissions:

- function setMinter()
- function setMintCap()
- function mint()
- function burn()
- function burnFrom()
- function pause()
- function unpause()

`DEFAULT_ADMIN_ROLE` can replace the bridge minter at any time, raise or lower the mint cap, and unpause the token. `PAUSER_ROLE` can freeze every transfer. The `minter` address has exclusive authority to mint new supply and to burn tokens (including `burnFrom` against any user that has granted allowance); this address is typically the bridge contract responsible for cross-chain message handling, and its own permission model (who can trigger its mint / burn entry points) is therefore part of this contract's trust boundary as well. The role holders listed above and the `minter` address all have the ability to influence how the protocol operates.

Recommendation

This finding describes the level of decentralization of the project. It is recommended to strengthen security and improve the degree of decentralization from the following aspects:

- It is recommended that holders of the privileged roles (`DEFAULT_ADMIN_ROLE`, `PAUSER_ROLE`) use multi-signature wallet addresses.

- For modification operations that affect protocol stability and key business parameters (such as `setMinter`, `setMintCap`, `unpause`), it is recommended to introduce a timelock to give off-chain monitoring a response window.
- The `minter` typically points to the bridge contract responsible for cross-chain message handling; the entry points that trigger its mint / burn flow should adopt the same level of decentralization protection. The bridge contract's permission model and upgradeability should be audited independently, because its security directly determines the supply security of this contract.

MAJOR

GLOBAL-02 | Uncleared Post-Rotateion Minter Allowances

Issue	Severity	Status
After Minter Rotation, the Previous Minter Can Still Drain User Funds via `transferFrom`	Major	Resolved

Description

setMinter(address newMinter)` only updates the `minter` state variable, so the previous minter loses access to the `mint` / `burn` / `burnFrom` entry points guarded by the `onlyMinter` modifier. However, **the ERC20 allowances that users previously granted to the old minter remain in storage**, and `MappingTokenV2` does not override the standard `transferFrom`. That function **has no `onlyMinter` or equivalent restriction**, so any spender holding an allowance — including the retired minter — can call it to move user tokens.

Attack path:
Step 1: User initiates a bridge-out flow
user.approve(MinterA, type(uint256).max) // bridge front-ends usually request infinite approval
MinterA.burnFrom(user, X) // normal bridge flow; almost the entire allowance remains

Step 2: Admin detects that MinterA is compromised / upgrade failed / has a bug
admin.setMinter(MinterB)
MinterA loses onlyMinter, but user → MinterA allowance is still in storage

Step 3: The attacker controlling MinterA directly calls:
MappingTokenV2.transferFrom(user, attacker, user balance)
↑ this is the standard ERC20 interface; onlyMinter does not gate it
↑ allowance is sufficient, _spendAllowance passes
↑ _transfer succeeds; user assets are siphoned away

Why this risk is particularly severe:

1. **The common motivation for calling `setMinter` is precisely that the previous minter has been compromised or has misbehaved.** In other words, at the moment the admin invokes `setMinter`, the old minter's trustworthiness is at its historical low — yet user approvals to it remain completely unchanged.
2. **Bridge front-ends typically default to requesting `type(uint256).max` infinite approval** (so users do not have to re-approve for each bridge transaction). Rotation + a user who has not actively revoked is equivalent to **leaving the full drain authority over the user's balance in the old minter's hands**.
3. **No privileged function on the V2 contract needs to be triggered at all**; the attack uses the standard ERC20 `transferFrom` path. Off-chain monitoring that relies on privileged-event alerts is therefore unlikely to fire.
4. **On-chain enumeration of "who has ever approved the old minter" is impossible**, so guiding users to revoke through documentation is an unreliable fallback there will always be silent addresses that never respond.

Recommendation

- Implement a mechanism in the contract that prevents the retired minter from spending residual allowance via `transferFrom`.
- In the user-facing integration documentation, explicitly instruct users that upon receiving the `UpdateMinter` event they should actively revoke the approval granted to the previous minter.

MEDIUM

GLOBAL-03 | Missing Context in Mint Cap Events

Issue	Severity	Status
Insufficient Event Context on Mint Cap Updates	Medium	Resolved

Description

`setMintCap(uint256 cap)` accepts any value, including a value below the current `totalSupply()`. The in-source comment explicitly documents this as an intentional mint-only emergency stop: transfers and burns continue to work, while new minting reverts.

However, the emitted event `UpdateMintCap(uint256 cap)` carries only the new cap value. It includes **neither** the previous cap **nor** `totalSupply()` at the time of the call. Subscribers cannot distinguish a routine cap adjustment from an emergency lowering of the cap below the current supply, which complicates monitoring and incident response.

Recommendation

- If easier off-chain detection is desired, consider extending the `UpdateMintCap` event with additional context such as the previous cap and the `totalSupply` at the time of the call, so that monitoring can directly distinguish a routine cap adjustment from an emergency stop.

MINOR

GLOBAL-04 | Zero-Amount Mint and Burn Permitted

Issue	Severity	Status
Zero-Amount Mint and Burn Permitted	Minor	Resolved

Description

The functions `mint(address, uint256)`, `burn(uint256)`, and `burnFrom(address, uint256)` do not reject `amount == 0`. A zero-amount call still emits an ERC20 `Transfer(address(0), to, 0)` or `Transfer(account, address(0), 0)` event, and `burnFrom(account, 0)` still goes through the allowance-check path against `account`. This is not exploitable but pollutes off-chain indexing pipelines and produces allowance-spending traces of unclear meaning.

Recommendation

- Add a guard at the entry of `mint`, `burn`, and `burnFrom`:
  `if (amount == 0) revert ZeroAmount();`.

Informational

GLOBAL-05 | Deprecated ERC20Permit Import

Issue	Severity	Status
Deprecated Import Path `draft-ERC20Permit.sol`	Informational	Resolved

Description

The contract imports `@openzeppelin/contracts/token/ERC20/extensions/draft-ERC20Permit.sol`. Starting from OpenZeppelin 4.9, the canonical path is `ERC20Permit.sol`; the file prefixed with `draft-` is only an 8-line forwarding shim whose entire body is `import "./ERC20Permit.sol";`. It is functionally identical to `ERC20Permit.sol` at runtime, and its header already marks it `// This file is deprecated.`. OpenZeppelin 5.x has removed the shim entirely.

Recommendation

Change the import to `@openzeppelin/contracts/token/ERC20/extensions/ERC20Permit.sol`. This change does not alter any runtime behavior.

Disclaimer

This report is based on the scope of materials and documentation provided for a limited review at the time provided. Results may not be complete nor inclusive of all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. A report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on the reports in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, we disclaim all warranties, expressed or implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. We do not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Appendix

Finding Categories

Centralization / Privilege
Centralization / Privilege findings refer to either feature logic or implementation of components that act against the nature of decentralization, such as explicit ownership or specialized access roles in combination with a mechanism to relocate funds.

Coding Style
Coding Style findings usually do not affect the generated bytecode but rather comment on how to make the codebase more legible and, as a result, easily maintainable.

Volatile Code
Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases that may result in a vulnerability.

Logical Issue
Logical Issue findings detail a fault in the logic of the linked code, such as an incorrect notion on how block. timestamp works.

Checksum Calculation Method

The "Checksum" field in the "Audit Scope" section is calculated as the SHA-256 (Secure Hash Algorithm 2 with digest size of 256 bits) digest of the content of each file hosted in the listed source repository under the specified commit.

The result is hexadecimal encoded and is the same as the output of the Linux “sha256sum" command against the target file.

About

DeHacker is a team of auditors and white hat hackers who perform security audits and assessments. With decades of experience in security and distributed systems, our experts focus on the ins and outs of system security. Our services follow clear and prudent industry standards. Whether it's reviewing the smallest modifications or a new platform, we'll provide an in-depth security survey at every stage of your company's project. We provide comprehensive vulnerability reports and identify structural inefficiencies in smart contract code, combining high-end security research with a real-world attacker mindset to reduce risk and harden code.

BLOCKCHAIINS

Ethereum
Cosmos
Eos
Substrate

TECH STACK

Python
Solidity
Rust
c++

CONTACTS

https://dehacker.io
https://twitter.com/dehackerio
https://github.com/dehacker/audits_public
https://t.me/dehackerio
https://blog.dehacker.io/

May 25th 2026